

Encoding Shaky Videos by Integrating Efficient Video Stabilization

Hua Huang[✉], *Member, IEEE*, Xiao-Xiang Wei[✉], and Lei Zhang, *Member, IEEE*

Abstract—This paper presents a novel video coding method by integrating video stabilization for shaky videos. By reusing the stabilized motion of feature points and geometric transformations, a better predictor can be established to replace the original motion vectors in the motion estimation stage of video coding. Then, these motion vectors are optimized based on statistics of the residuals between the stabilization predictor and the standard one to improve the efficiency of motion search. As a result, our method brings much less computational cost to motion estimation than encoding the stabilized frames separately (e.g., 24% for the enhanced predictive zonal search algorithm, 17% for the unsymmetrical-cross multi-hexagon-grid search algorithm), while the Bjontegaard distortion (BD) bit rate and BD-peak signal-to-noise ratio still have the comparable performance with multiple quantization parameters. Specially, the implementation of our integrative system based on x264 is very fast and of low latency, by which full HD videos can be simultaneously stabilized and encoded with more than 30 frames per second in the fastest mode, even on the mobile platform. The experiments on a variety of shaky videos demonstrate the potential of our method in terms of effectiveness and efficiency.

Index Terms—Video coding, video stabilization, motion vector.

I. INTRODUCTION

VIDEOS captured by cameras mounted on the mobile devices (e.g., mobile phones, drones, camcorders) often exhibit shaky motion subject to mechanical vibration. The shakiness induced by camera movement not only incurs a terrible experience when viewing the video content, but also increases the computational cost and bit rate in video coding [1], [2]. This degeneration is quite unfavorable because it overloads video storage and transmission. While mobile devices usually suffer the limited battery power and network bandwidth, video coding must adapt to shaky videos.

It has been proven that video stabilization can greatly reduce bit rate in encoding shaky videos [1]. Therefore, combining video stabilizer with video encoder seems to be promising in improving video coding and obtaining stabilized videos simultaneously. Actually, both video coding and video stabilization depend in a large part on motion vector search in

motion estimation, while video stabilization further applies motion smoothing for compensating the shakiness. Moreover, motion vectors generated by video stabilization bear many similarities with motion vectors of macroblocks in video encoder. Thus, reusing motion vectors of video stabilization is pretty applicable to the integration of video stabilizer with video encoder.

However, both video coding and video stabilization are compute-intensive tasks. For the purpose of real-time and low-latency, existing integration mainly employs low-pass filter to compensate the two-dimensional (2D) shaky motion between adjacent frames for video stabilization [2], [3]. These methods do reduce the bit rate in video coding, but the performance of coding efficiency and stabilization quality is not within the reach of desideratum. Especially, it is known that video stabilization based on low-pass filter cannot well deal with complex motion [4], [5], e.g., with large parallax or shakiness amplitude. Thus, the encoded video still exhibits apparent shakiness in the display, which suggests more room to adapt the integration with more sophisticated video stabilization methods. Recent years have witnessed a rise in high-quality video stabilization methods, e.g., using three-dimensional (3D) scene structure for motion estimation [6], [7] or 2D spatial constraints for motion smoothing [5], [8]–[10], even the one oriented for panoramic video stabilization [11]. Although these methods enable plausible stabilization results, the high computational cost makes them prohibitively expensive in the integration with video coding. In this paper, a light-weight yet high-quality geodesic video stabilization method [12] is employed in video coding, which makes a good progress on the efficiency of some key components in video encoder. Generally, our work can be seen as an extension of our previous work on video stabilization to video coding. Although the stabilization technique is borrowed from [12], the motivation and application domain are different. We want to make a graceful integration of the stabilization and encoding that finally benefits the efficiency of video coding.

The main contribution of our work is a one-shot integration of the stabilized motion into the video coding, rather than encoding the stabilized frames afterwards. As the novelty of our one-shot integration, we reuse and optimize stabilization motion vectors of (sub-)macroblocks in the stage of motion search. This greatly reduces the computational cost of motion search, while having favorable R-D performance comparable to encoding the stabilized frames separately. The experiments show the appealing performance of our integrative system on video coding efficiency besides of stabilization effectiveness.

Manuscript received February 10, 2018; revised April 2, 2018; accepted April 30, 2018. Date of publication May 7, 2018; date of current version May 3, 2019. This work was supported by the National Natural Science Foundation of China under Grant 61772069 and Grant 61425013. This paper was recommended by Associate Editor G. Valenzise. (*Corresponding author: Hua Huang.*)

The authors are with the Beijing Key Laboratory of Intelligent Information Technology, School of Computer Science, Beijing Institute of Technology, Beijing 100081, China (e-mail: huahuang@bit.edu.cn; 583288152@qq.com; leizhang@bit.edu.cn).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCSVT.2018.2833476

1051-8215 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

II. RELATED WORKS

A. Video Coding With Stabilization

Video coding aims at compressing a video to a reasonable size by removing spatial and temporal redundancies in the raw video data. Since motion-compensated prediction can remove the temporal redundancies efficiently [13], the use of stabilized motion vector is straightforward to improve video encoding by stabilization. This can reduce the displacement of macroblocks between the reference frame and the predicted one in the current frame.

The motion vectors can be generated by low-pass filtering the trajectories through the frame sequence in the block-based stabilizers, which is afterward used in the motion estimation stage of video coding [1], [3]. Alternatively, the motion vectors generated from video encoder can also be used to stabilize shaky videos [2]. Consequently, shaky videos are converted to bitstream with less shakiness or shakiness-free. However, these methods have little improvement in the coding efficiency, because they still need a complete motion search procedure to guarantee that each macroblock is assigned with a corresponding motion vector. In fact, there is even a slight increase in the computational cost compared to the ordinary video encoder, and the stabilization quality is not satisfactory.

A few encoding methods resort to an expert device or application context to improve the efficiency of both coding and stabilization. Chen *et al.* [14] adopt accelerometer and compass to remove shakiness caused by rotation, which immediately produces stabilized frames as the input of encoding. Ding *et al.* [15] model background by global motion compensation to encode shaky surveillance videos. These methods can reduce the computational cost of video coding as well as enhance the video quality, but are not applicable for general use.

B. Video Stabilization

Video stabilization resorts to warping the frames to obtain visually smooth inter-frame motion. According to the dimensionality of motion representation and smoothness constraints used in motion estimation and smoothing, video stabilization methods can be roughly classified into three categories: 2D methods, 3D methods, and 2.5D methods.

2D methods estimate the camera motion based on planar displacement vectors or transformations between adjacent frames [9], [10], [16], [17]. Then, the smoothed motion is computed by using Gaussian filtering [16], [18], [19], least-square fitting [20] or L_1 -norm optimization [4], [21]. 2D methods have low computational cost, but suffer bad stabilization quality that cannot well handle complex motions with large parallax or severe shakiness. 3D methods opt for reconstruction of camera motion in the 3D space for motion estimation, which can be realised by applying structure-from-motion (SfM) [6], [7] or sensors [22], [23]. Obviously, 3D methods enable decent stabilization quality, but they are usually not such robust in practice that is hard to achieve real-time encoding.

As a trade-off of 2D methods and 3D methods, 2.5D methods use spatial information to improve motion

smoothing without costly 3D reconstruction, e.g., using space-time constraints based on free-form curve fitting [24], low-rank constraints [5], or spatially variant transformations [8], [25]. Generally, this kind of methods achieves a good balance on computational cost and stabilization quality. However, they usually come at higher cost than 2D method and might fail to remove severely shaky motion. Unlike previous works just doing video stabilization, our method further attempts to reuse the stabilized motion in the intermediate stage of motion estimation of video encoder, which would generate much less computational cost than directly encoding the stabilized frames after stabilization.

Considering the task of video coding, 2D methods stand out for its inexpensive computation despite degradation in stabilization quality. Fortunately, Zhang *et al.* [12] recently propose a distinctive method of geodesic video stabilization in transformation space, which possesses the merit of 2D methods but enables high-quality stabilization. Meanwhile, more accurate motion vectors can be computed from transformations that are derived in stabilization. Hence, we borrow this stabilization method to do video coding in this paper, which presents a more feasible integration of video coding and stabilization.

III. BACKGROUND

We borrow the technique of geodesic video stabilization [12] to generate the stabilization motion for the usage in video coding. So this section gives a brief introduction on the stabilization method to make it self-contained in this paper. Basically, the geodesic video stabilization takes the inter-frame motion represented by geometric transformations as a path on a high-dimensional manifold, and then obtains the smoothed path by optimization on the manifold.

The geometric transformation (e.g., homography) is computed based on the feature correspondences between adjacent frames, denoted by $F_t : I_{t-1} \rightarrow I_t$, where I_{t-1} and I_t represent the two adjacent frames in the frame sequence. In mathematics, the set of specific transformations is a *Lie group*, which is a topological group as well as a differential manifold [26]. Thus, the motion of the shaky video can be represented by a path on the manifold, and denoted by the concatenation of inter-frame transformation as $P_t = F_1 \cdot F_2 \cdots F_t$. Then, the smoothed path boils down to the geodesics by optimizing the integral energy function as below

$$\tilde{P}_t = \operatorname{argmin}_{P_t \in \mathcal{G}} \int_{t_0}^{t_1} \left\langle \frac{dP_t}{dt}, \frac{dP_t}{dt} \right\rangle dt \quad (1)$$

where $\mathcal{G}$ is a specific *Lie group* manifold, and $\langle \cdot, \cdot \rangle$ is the inner product defined in the tangent plane of the manifold. Here, the tangent plane is spanned by all the tangent vectors at a point (the first-order derivatives), which characterizes the velocity along the motion path.

Fortunately, we have an explicit expression for the solution of Eq.(1), i.e., $\tilde{P}_t = \operatorname{Exp}_{P_{t_0}}(\tilde{t} \operatorname{Log}_{P_{t_0}}(P_{t_1}))$, where $\operatorname{Exp}(\cdot)$ is the Riemannian exponential function and $\operatorname{Log}(\cdot)$ is the Riemannian logarithm function [27], and $\tilde{t} = (t - t_0)/(t_1 - t_0)$ indicates the moment inbetween the start and end frames. This enables a very fast motion smoothing in the stabilization.

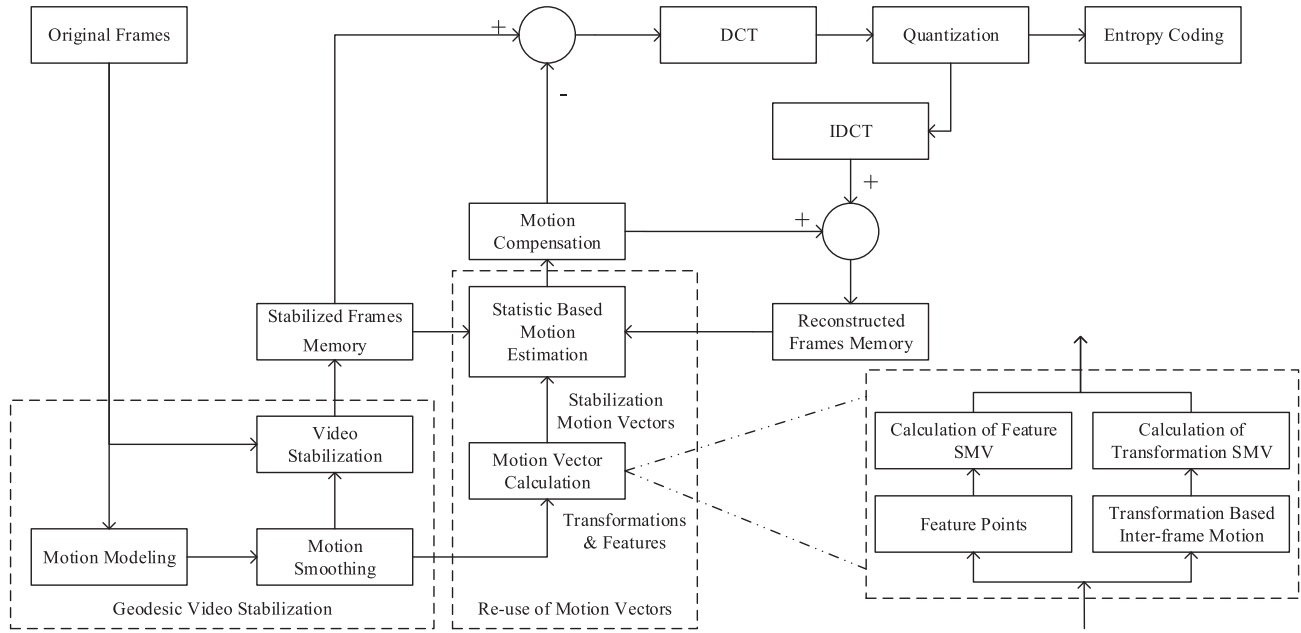


Fig. 1. System overview. The stabilized frames and transformations are generated from geodesic video stabilization [12]. Motion vectors from feature points and transformations are reused in motion vector calculation and motion estimation of video encoder.

Then for each frame I_t , we have its compensation transformation $B_t = \tilde{P}_t \cdot P_t^{-1}$, such that the transformed frame $\tilde{I}_t = B_t I_t$ follows the smoothed path defined by Eq.(1). Thus, we obtain the stabilized frame sequence. Next, the stabilization motion will be exploited to generate motion vectors for video coding.

IV. STABILIZATION MOTION VECTORS

A key idea behind our integration with video coding is to make use of the inter-frame transformations and displacements of feature points in the stabilization, to generate more reliable motion vectors, namely stabilization motion vectors (SMVs). These motion vectors can be directly used in the motion estimation stage of video coding as shown in Fig. 1.

A. System Overview

We start with an overview of our integration system (see Fig. 1). Stabilized frames and frame transformations are generated after raw video frames being stabilized by the geodesic video stabilizer, which are subsequently sent to video encoder. Here, H.264/AVC standard [28] is chosen as the implementation of our encoder. Directly encoding the stabilized frames can reduce bit rate caused by shaky motion, but cannot save any computational cost of the ordinary encoder. So we reuse motion vectors induced by both feature points and transformations from stabilization, such that every macroblock has its own motion vector (see Section IV-B and Section IV-C). This adaption enables better prediction to facilitate motion estimation (see Section V-A).

Moreover, we find out that motion vectors of a macroblock bear a close resemblance to its sub-macroblocks based on statistical analysis. So we further adapt motion estimation for motion search optimization by reusing stabilization motion vector of the macroblock to expedite the operation of its

sub-macroblocks. This can greatly reduce the computational cost in the case of sub-macroblocks taking more time than the macroblock on motion vector prediction (see Section V-B).

B. Transformation Based Inter-Frame Motion

We can directly calculate the SMVs based on the stabilized frames, but this does not benefit the computational efficiency as aforementioned. Actually, in the process of geodesic video stabilization, we already have the geometric transformation F_t based on the feature correspondences between adjacent frames. However, there is a little difference of the inter-frame motion between video stabilization and video coding: only transformation of adjacent frames is involved in video stabilization, while video coding needs additional inter-frame motion between the current and the reference frames that might not be adjacent. So it is necessary to obtain the transformations of non-adjacent frames for the SMV calculation. Fortunately, we can borrow those transformations at hand in video stabilization to obtain the desirable transformations in video coding.

Concretely, we assume the current frame is I_t , and its reference frame is I_{t-m} , where $m \geq 1$. The transformation between these two frames of the original video can be immediately obtained as $T_{t,t-m} = P_{t-m} P_t^{-1}$. Furthermore with the known compensation transformations of B_t and B_{t-m} respectively, the transformation between $\tilde{I}_t$ and $\tilde{I}_{t-m}$ can be easily computed as follows:

$$\tilde{T}_{t,t-m} = B_t T_{t,t-m}^{-1} B_{t-m}^{-1} = B_t P_t P_{t-m}^{-1} B_{t-m}^{-1} \quad (2)$$

where all the transformations have been derived in video stabilization (see Fig. 2). Thus, we obtain the inter-frame motion based on the transformation between the current frame and the reference frame.

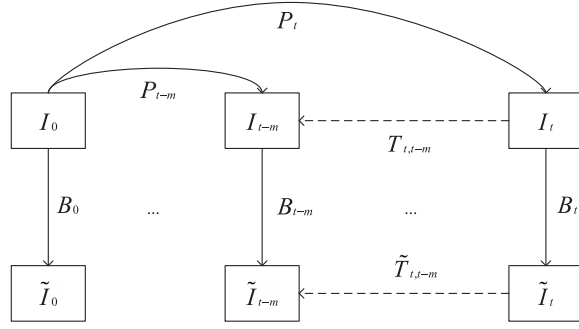


Fig. 2. The inter-frame motion induced by transformation between non-adjacent frames (dashed arrows) can be computed by the motion path of the original video and compensation transformations (solid arrows).

C. Calculation of Stabilization Motion Vector

In the ordinary H.264/AVC, each frame is partitioned into a set of macroblocks of size 16×16 before encoding. According to *tree-structured motion compensation strategy*, each macroblock can be further partitioned into smaller sub-macroblocks with the size of 16×8 , 8×16 , 8×8 , 8×4 , 4×8 and 4×4 . These partitions are based on rate-distortion optimization (RDO) principle [29]. For encoding video frames, every block (including 16×16 and smaller sizes) is assigned with its own motion vector separately. In our integration, we fully exploit the motion from video stabilization to calculate the SMV as the initial motion vector assignment of the macroblock and its sub-macroblock as used in the next section. Typically, we have two kinds of SMVs: *feature SMV* and *transformation SMV*.

1) *Feature SMV*: If a block already has feature points that are used for inter-frame transformation computation in video stabilization, we can directly calculate the block's motion vector, namely *feature SMV*, by the displacement of the feature correspondences after stabilization. Typically, these feature points form continuous trajectories across a bunch of successive frames, which enables the calculation of SMVs between non-adjacent frames.

We assume there are n_i feature correspondences within the i -th block of the current and the reference frames, denoted by the pairs of feature points $\{(x_k^{cur}, y_k^{cur})\}_{k=1}^{n_i}$ and $\{(x_k^{ref}, y_k^{ref})\}_{k=1}^{n_i}$ respectively. Then, the displacement between a pair of matching feature points is defined by

$$\mathbf{d}_k(dx_k, dy_k) = (x_k^{ref} - x_k^{cur}, y_k^{ref} - y_k^{cur}) \quad (3)$$

whereupon we obtain n_i displacement vectors for the block. To obtain the final stabilization motion vector $\text{SMV}_i(vx_i, vy_i)$ for the coding, we simply compute the average of all the displacement vectors within the block, i.e.,

$$\text{SMV}_i(vx_i, vy_i) = (\lfloor \frac{1}{n_i} \sum_{k=1}^{n_i} dx_k \rfloor, \lfloor \frac{1}{n_i} \sum_{k=1}^{n_i} dy_k \rfloor) \quad (4)$$

where the operator $\lfloor \cdot \rfloor$ maps a real number to the greatest preceding integer. Thus, we obtain the *feature SMV* for the block that has its own feature points, which actually depicts the displacement from the geometric center C_i^{cur} of the feature points in the current frame to C_i^{ref} in the reference frame (see Fig.3(a)).

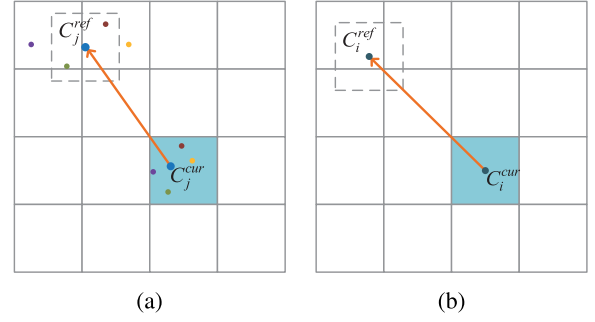


Fig. 3. (a) The *feature SMV* is calculated by the displacement of the geometric centers of feature points (the colored dots). (b) The *transformation SMV* is calculated by the transformation of the geometric center of the block.

2) *Transformation SMV*: As video coding needs motion vector of each block, we have to confer motion vectors for the blocks without feature points. This can be done by using transformations obtained in video stabilization, to generate the so-called *transformation SMV*.

Concretely, given the j -th block with no feature points in the current frame, we have the transformation to its corresponding reference frame as $\tilde{T}_{cur,ref}$ defined by Eq.(2). We define the geometric center of a macroblock as the intersection of the two diagonal lines of the corresponding square. Then, this transformation delivers the geometric center (x_j^{cur}, y_j^{cur}) of the current block to a new position (x_j^{ref}, y_j^{ref}) in the reference frame, i.e.,

$$\begin{bmatrix} x_j^{ref} \\ y_j^{ref} \\ 1 \end{bmatrix} = \tilde{T}_{cur,ref} \begin{bmatrix} x_j^{cur} \\ y_j^{cur} \\ 1 \end{bmatrix} \quad (5)$$

where the positions are represented by homogeneous coordinates. Then, we calculate an overall displacement of the current block with respect to the transformation as follows:

$$\text{SMV}_j(vx_j, vy_j) = (\lfloor x_j^{ref} - x_j^{cur} \rfloor, \lfloor y_j^{ref} - y_j^{cur} \rfloor) \quad (6)$$

where $\lfloor \cdot \rfloor$ is defined as in Eq.(4). Thus, we obtain the *transformation SMV* of the block without feature points in the current frame, which actually depicts the displacement of the geometric center C_j^{cur} of the current block to C_j^{ref} of the reference block (see Fig.3(b)).

Remark: Due to the re-use of motion information in video stabilization, the motion vectors involved in encoding videos can be fast calculated with extremely low computational cost. Actually, *feature SMV* gives the local motion of the block, while *transformation SMV* supplies the motion of the block without features by using the global transformation of the frame. Although the SMVs might differ from those motion vectors obtained by an ordinary encoder on the stabilized videos, they enable a good initialization for further optimization. The optimization on motion estimation will be elaborated in the next section.

V. STATISTICS-BASED OPTIMIZATION ON MOTION ESTIMATION

Motion vector prediction and motion search are two important components in the motion estimation stage of video

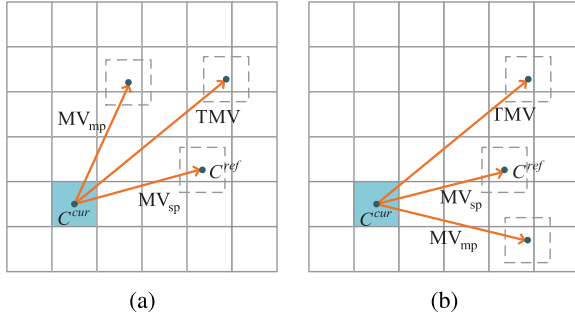


Fig. 4. The motion vectors generated by the *median predictor* (MV_{mp}), *stabilization predictor* (MV_{sp}) and the *standard motion vector* (TMV). (a) shows a case that MV_{sp} and MV_{mp} are almost equally close to TMV . (b) shows a case that MV_{sp} is much closer to TMV than MV_{mp} .

coding. Its efficacy can be boosted by further improving the accuracy of predictor, as well as reducing the complexity of motion search steps. While the difference between SMVs and ordinary motion vectors does influence the bit rate and video quality after video coding, we put forward a SMV predictor with statistics-based motion vector optimization, by which video coding achieves a trade-off among bit rate, video quality and computational efficiency on motion estimation.

A. Stabilization Motion Vector Predictor

In an ordinary video encoder like H.264/AVC, the search of motion vector starting at an initial point is generally produced by a specific predictor, e.g., *median predictor* and *uplayer predictor*. Obviously, the distance between the initial point and the final best-matched point determines the efficiency of the predictor, i.e., the times of the motion search. Hence, we can employ the stabilization motion vectors generated in Section IV as a new kind of predictor, namely *stabilization predictor*.

But unlike video coding for the minimum redundancy of blocks in prediction, video stabilization does not seek for minimizing the matching cost of blocks in motion smoothing and compensation. Consequently, the *stabilization predictor* does not always generate the optimal motion vector as shown in Fig.4(a). Here, we define the final motion vectors generated by using common motion search algorithms (e.g., EPZS and UMHS as stated in Section VI) as standard motion vectors ($TMVs$). Besides, motion vectors generated by the *median predictor* are defined as MV_{mp} . To avoid more expensive motion search, only *stabilization predictor* that is better than existing predictors like *median/uplayer predictor* is used to search for the final motion vector (see Fig.4(b)).

Therefore, we first compute the RDO cost of different predictors, and then choose the best one with the minimum cost as the initial point of motion search.

Remark: The advantage of using *stabilization predictor* is that it can be directly built by SMV from video stabilization. However, the reduction of computation induced by *stabilization predictor* is rather limited in the whole process of video coding, because like other predictors, it is usually not accurate enough to avoid motion search completely for macroblocks. To further benefit computational efficiency, we need to expand

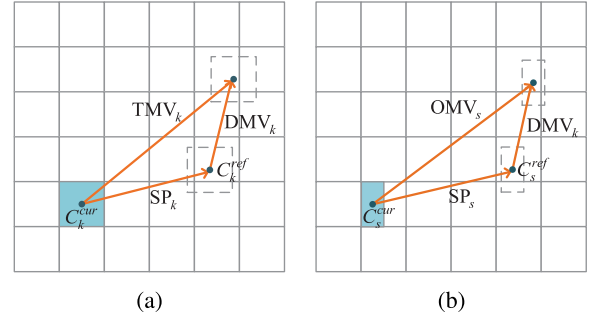


Fig. 5. Relations of DMV and OMV. (a) shows the calculation of DMV. (b) shows that DMV is used in motion search of a sub-macroblock.

capabilities of these predictors on optimizing motion estimation for the sub-macroblocks in the next section.

B. Statistics-Based Motion Vector Optimization

In the H.264/AVC encoder, *tree-structured motion compensation strategy* forces motion search working in a specific order. Concretely, a macroblock of 16×16 size is processed and then followed by its sub-macroblocks with the smaller size, e.g., 16×8 , 8×16 and 8×8 . Actually, the neighboring region of the *stabilization predictor* in the reference macroblock is similar to the neighboring region in the current macroblock, which usually enables smaller cost the in motion vector prediction for the sub-macroblocks of the 16×16 macroblock.

Concretely, the disparity between two motion vectors must be defined at first. If the *stabilization predictor* of the k -th block is better (with a smaller RDO cost) than other predictors, we denote the predictor by $SP_k(px_k, py_k)$. We denote the standard motion vector with the lowest cost for the final coding by $TMV_k(vx_k, vy_k)$. Then, the disparity of these two motion vectors can be defined as follows:

$$DMV_k(dx_k, dy_k) = (vx_k - px_k, vy_k - py_k) \quad (7)$$

which characterizes the deviation of the motion vector of *stabilization predictor* from the standard one (see Fig.5(a)). Then, we can continue motion search for the motion vector of the macroblock (see Algorithm 1). It should be noted that if the RDO cost of *stabilization predictor* is larger than the classic predictors like *median predictor*, we choose the search strategy of the ordinary encoder and set DMV_k to be *null*.

Algorithm 1 Motion Search for 16×16 Macroblock

Ensure: MV_{sp} and MV_{mp}
 compute RDO cost of MV_{sp} and MV_{mp}
if $RDO_{sp} \leq RDO_{mp}$ **then**
 set MVP_{sp} as initial point
 search for TMV_k
 $DMV_k = TMV_k - MV_{sp}$
else
 set MV_{mp} as initial point
 search for TMV_k
 $DMV_k = null$
end if

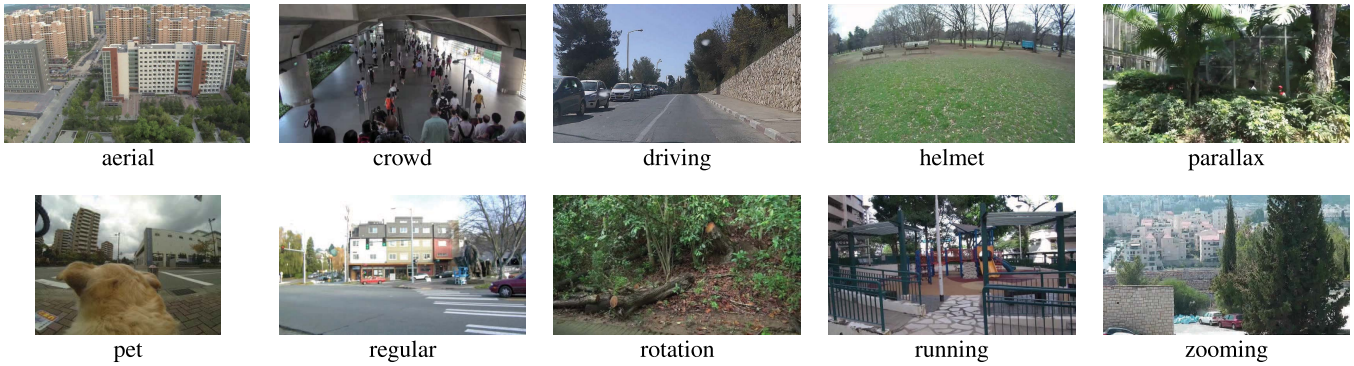


Fig. 6. The dataset contains 10 categories of shaky videos.

Considering the sub-macroblocks in the 16×16 macroblock, we adopt the above motion disparity to assist the motion search of those sub-macroblocks. Suppose the *stabilization predictor* of a sub-macroblock is $SP_s(px_s, py_s)$, then we can grant it an optimized motion vector as

$$OMV_s(vx_s, vy_s) = (px_s + dx_k, py_s + dy_k) \quad (8)$$

which inherits the motion disparity (dx_k, dy_k) from the macroblock (see Fig.5(b)). Although the derivation of Eq.(8) differs the classic formulation of motion vector, it has good coincidence and commendable accuracy for motion estimation as evidenced by the statistics in the experiments in Section VI-A. For this reason, we name it statistics-based motion vector optimization for the sub-macroblock.

Algorithm 2 Motion Search for Sub-Macroblock

Ensure: MV_{sp} , MV_{mp} and DMV_k

if $DMV_k \neq null$ **then**

$OMV_s = MV_{sp} + DMV_k$

 do selective search

else

 set MV_{mp} as initial point

 search for OMV_s

end if

It should be noted that there is a small displacement between geometric centers of the 16×16 macroblock and its sub-macroblock, which probably produces a deviation between OMV_s and the ideal motion vector with the minimum cost. To avoid this slight deviation, we further adopt a *selective search strategy*, in which some effective search strategies are used, e.g., small diamond search (SDS) or square search [30]. The option on the strategy will be determined according to the results of our experiments as presented in Section VI-B. Thus, we can perform motion search for the optimized motion vector of the sub-macroblock (see Algorithm 2).

VI. EXPERIMENTS

We have implemented the integrative system of video stabilization with video coding based on the H.264/AVC reference software [31] (JM version 19.0), and various experiments have been done to evaluate the performance

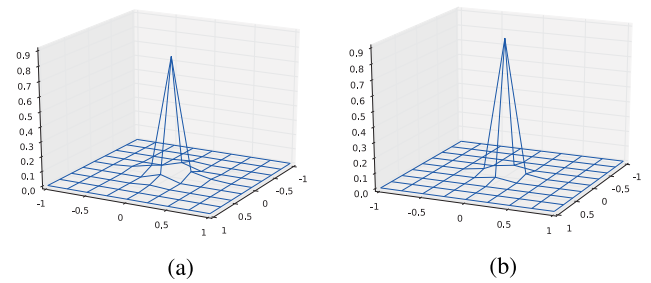


Fig. 7. Statistics of the difference between TMV_s and OMV_s in the unit of quarter-pixel based on EPZS (a) and UMHS (b). The x - and y -coordinate denote the difference of the two motion vectors, and the z -coordinate is the percentage with respect to the number of pixels.

of our system. The encoding procedure is based on a set of default parameters, of which we reset some ones like multiple quantization parameters (QP), in 22, 27, 32 and 37) and with the smallest block size of 4×4 , as well as early-termination-applied motion search algorithm without B-frame setting (including unsymmetrical-cross multi-hexagon-grid search algorithm (UMHS) [32] and enhanced predictive zonal search algorithm (EPZS)).

We did the experiments on both a desktop computer and an ARM-based mobile device. Bjontegaard distortion bit rate (BD-rate) and Bjontegaard distortion peak signal-to-noise ratio (BD-PSNR) [33] are used as an objective measurement of the encoded video quality, and we also use the average elapsed time of motion estimation to represent the computational cost. Both real data and synthetic data are used for performance evaluation in this section.

A. Accuracy of Motion Estimation for Sub-Macroblock

We have calculated OMV_s of the sub-macroblock by Eq.(8) in Section V-B based on the statistics of motion difference in encoding videos. Here, we make a test on the dataset as shown in Fig. 6, to verify the accuracy based on the statistical results. This dataset contains 328 clips of real shaky videos in the daily life (see the details of the dataset in Section VI-C).

Concretely, we assume TMV_s is the standard motion vector of the sub-macroblock produced by using *stabilization predictor* SP as the initial point based on UMHS and EPZS respectively. Then, the difference (in pixel) between TMV_s

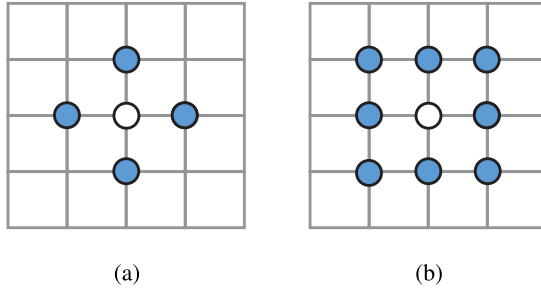


Fig. 8. The search template of small diamond search (a) and square search (b). The white and blue points represent the current position and position being searched respectively.

and OMV_s is computed as shown in Fig. 7. Specially, we use the 3D line chart to represent the frequency distribution of the difference in the unit of quarter-pixel. It can be seen that more than 85% of the difference located at the origin (0, 0), which means the high accuracy of using OMV_s as the motion vector in motion estimation. Moreover, if the SDS strategy is attached to motion estimation, the ratio can rise to 95% near the origin. Based on these results, it is believed that our statistics based optimization enables good motion vectors, by which a substantial reduction of computation in motion search of sub-macroblocks is also within reach. So we use motion vectors by the statistics-based optimization in this paper.

B. Selective Search Strategy

There are two options after OMV_s being calculated by Eq.(8). One is to directly use these OMV_s as the final motion vectors, and the other one is to further implement a local search like small diamond search (see Fig. 8(a)) or square search (see Fig. 8(b)) for better performance. To balance the computation and the performance, we implemented those three strategies and tested them on the videos in the dataset.

We evaluated these strategies with three measurements: PSNR, bit rate and computational cost. To illustrate the difference better, we adopt the percentage variation of the measurements. Regarding the direct OMV as the baseline, the percentage variation of PSNR is defined as the percentage of $\Delta(\text{PSNR})$ within the range of baseline PSNR. The percentage variations of bit rate and computational cost are also defined as above. Fig. 9 shows the testing results on the dataset. It can be seen that both bit rate and PSNR of these strategies are very close to each other with percentage variations lower than 0.5%. Besides, the computational cost of the direct OMV obtains a significant reduction compared to the other two strategies with 6.4% and 11.7% respectively. So we recommend the strategy of direct OMV in practice for the best trade-off, which can greatly reduce the computation yet maintain a good performance on bit rate and PSNR in video coding.

C. Experiments With Real Data

We firstly test our system with the dataset of real shaky videos. The dataset is composed of 328 short clips of 10 categories: *aerial*, *crowd*, *driving*, *helmet*, *parallax*, *pet*, *regular*, *rotation*, *running* and *zooming*, which covers the

most types of shaky motions in daily life (see the sampled frames in Fig. 6). For example, the videos in the category of *helmet* and *pet* are captured by a camera mounted on a helmet or placed on the back of a pet. Some of the clips are from the online dataset published in [8], and the others are collected by ourselves that can be downloaded from our webpage.¹

All the videos in the dataset are encoded by using our integrative method with multiple QPs (22, 27, 32 and 37), and obtain the corresponding R-D performance and computational cost of each video. To demonstrate the superiority of our method, we also make comparisons with the methods of shaky-encoder and stabilized-encoder. Here, shaky-encoder means directly encoding the original shaky videos, while stabilized-encoder means encoding the output videos of stabilization. Next, we give the detailed analysis of these results.

1) *R-D Performance*: Fig. 10 shows the R-D curve by using the three methods (our method, shaky-encoder, and stabilized-encoder). Here, we adopt the bit-rate and PSNR produced by JM encoder to demonstrate the R-D performance. Here, the PSNR values are computed by taking the frames before encoding as the reference, because the ground truth stable videos are not available for the real data. For example, we have $\text{PSNR}(I_t^c, I_t)$ for shaky-encoder and $\text{PSNR}(\tilde{I}_t^c, \tilde{I}_t)$ for our method and stabilized-encoder, where I_t^c and $\tilde{I}_t^c$ denote the encoded frames. It should be noted that in theory, our method should have the same PSNR values with stabilized-encoder. In fact, as shown in Fig. 10, the R-D performance of our method does have almost the same values with the stabilized-encoder, and both of them obtain better performance than shaky-encoder especially in high bit ranges. This is because the distance of matched macroblocks in lower QPs is larger than the one in higher QPs, which generates larger bit rates of bitstream in lower QPs.

To present a standard evaluation of the compression effectiveness, we also list the BD-PSNR and BD-rate in Table I. Here, the values of BD-PSNR are computed by taking the results of shaky-encoder as the reference. Generally, the BD-PSNR performance gains about 1.09dB on average and peaks up to 1.74dB by our method. The increase in BD-PSNR is mainly due to the shakiness compensation of video stabilization, and the amount depends on the degree of removed shakiness. Fig. 11 shows two videos from the category of *regular*, which have different degrees of shakiness as illustrated by the trajectories of frame centers. The two videos gain 0.59dB and 0.28dB after encoding by our method. We notice in this example, the first one suffers from massive unpredictable large translation (the serrations), while the second one has just a slight shakiness. Actually, the elimination of larger shakiness results in a smaller residual between the original frames and the encoded frames, such that the BD-PSNR becomes larger after video coding as shown in Fig. 11(a).

As an example, Fig. 12 shows two frames sampled from the original and stabilized videos in the category of *running*. It can be seen that there are more macroblocks of I mode in the shaky frame than the stabilized frame (see Fig. 12(b)(e)). Besides,

¹<https://zhangleibit.github.io/download/stabcoding/testdata.html>

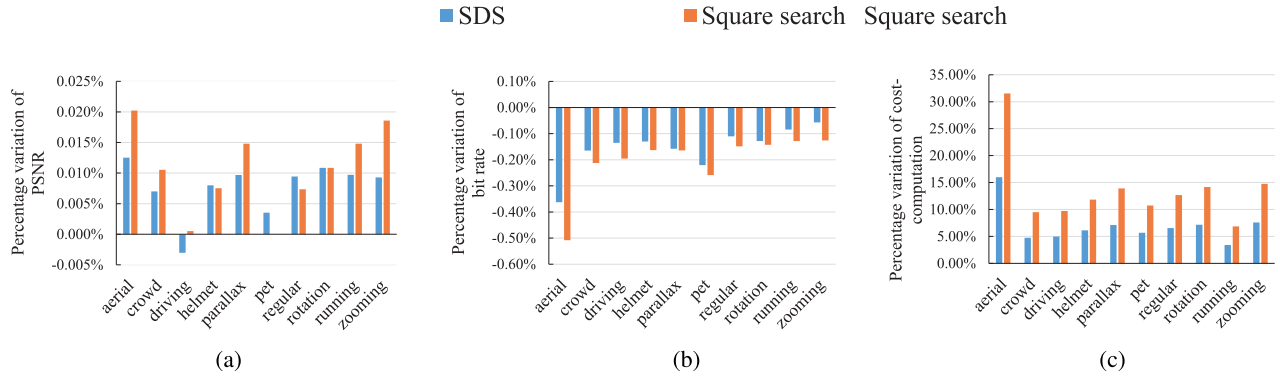


Fig. 9. The coding results of the dataset in terms of PSNR, bit rate and computational cost by applying the search strategies of direct OMV, small diamond search and square search respectively. The values of direct OMV are used as the anchor to show percentage variations of other strategies. Here, the positive variations mean increase and the negative variations mean decrease. (a) PSNR. (b) Bit rate. (c) Computational cost.

TABLE I

THE BD-RATE AND BD-PSNR PERFORMANCE OF OUR METHOD AND STABILIZED-ENCODER WITH SHAKY-ENCODER AS THE REFERENCE

category	stabilized-encoder with EPZS		our method with EPZS		stabilized-encoder with UMHS		our method with UMHS	
	BD-rate	BD-PSNR	BD-rate	BD-PSNR	BD-rate	BD-PSNR	BD-rate	BD-PSNR
aerial	-11.55%	0.46dB	-11.21%	0.45dB	-12.37%	0.51dB	-11.66%	0.47dB
crowd	-15.16%	0.80dB	-13.70%	0.71dB	-19.10%	1.06dB	-17.54%	0.96dB
driving	-21.62%	1.05dB	-19.87%	0.95dB	-29.64%	1.53dB	-27.22%	1.37dB
helmet	-15.08%	0.88dB	-13.77%	0.79dB	-25.80%	1.54dB	-24.88%	1.50dB
parallax	-14.64%	0.93dB	-13.91%	0.88dB	-12.11%	0.82dB	-11.35%	0.76dB
pet	-23.15%	1.23dB	-21.61%	1.14dB	-26.97%	1.50dB	-25.36%	1.39dB
regular	-17.21%	0.87dB	-15.85%	0.79dB	-27.73%	1.54dB	-26.66%	1.46dB
rotation	-21.28%	1.01dB	-20.02%	0.94dB	-24.02%	1.19dB	-22.78%	1.11dB
running	-21.96%	1.23dB	-20.97%	1.17dB	-28.53%	1.70dB	-27.64%	1.64dB
zooming	-30.86%	1.55dB	-30.10%	1.50dB	-34.43%	1.81dB	-33.42%	1.74dB
Avg.	-19.24%	1.00dB	-18.10%	0.93dB	-24.07%	1.32dB	-22.85%	1.24dB

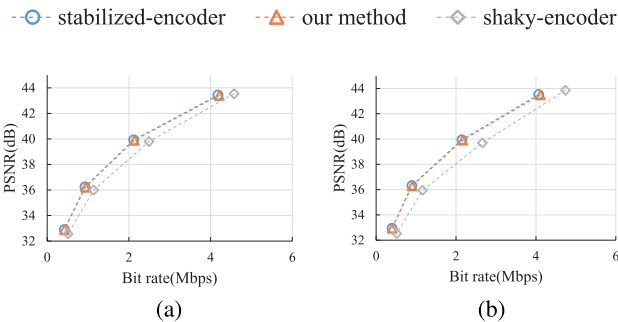


Fig. 10. The R-D curves of the shaky-encoder, stabilized-encoder and our method where video clips are encoded by JM encoder using (a) EPZS and (b) UMHS algorithm.

the motion vectors in the original frame are much longer (see Fig. 12(c)(f)), which means a longer distance between the current block and the best-matched block.

As for the BD-rate, both our method and stabilized-encoder have the average reduction about 20% with respect to the original shaky videos. The reasons lie in two aspects. Firstly, in the original shaky video, the RDO cost between the current block and the best-matched block is too large to encode, such that the prediction of I mode is invoked to restrict the bits of the encoded video. Because stabilization can reduce the cost of motion vector and residual by removing the shakiness, it results in an increase on the number of blocks in P-SKIP

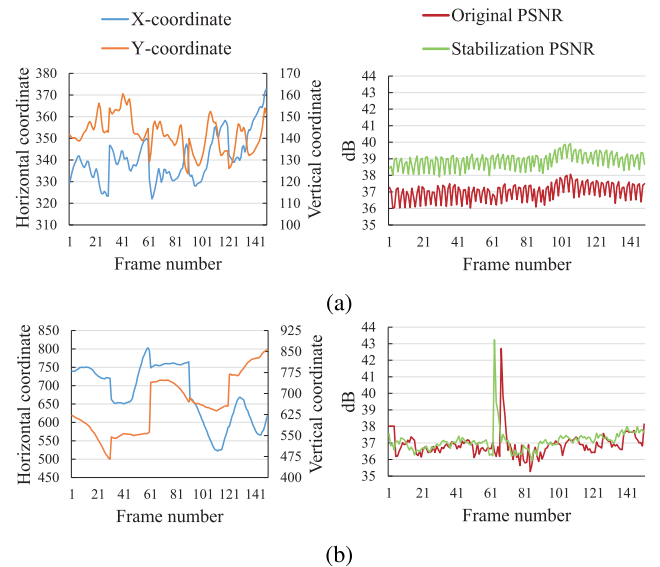


Fig. 11. The trajectories of the frame centers of two videos in the category of *regular* (left) and their PSNR after video coding with stabilization by our method (right).

mode. Secondly, motion vectors of blocks that are encoded in P mode contribute bits of video, while the distance between the current block and the best-matched block increases due to the severe shakiness. Overall, our method can achieve comparable

TABLE II

THE PERCENTAGE OF TIME SAVING (TS) BY STABILIZED-ENCODER AND OUR METHOD, BOTH COMPARED TO SHAKY-ENCODER. WE EVALUATE THE ELAPSED TIME BY CALCULATING THE TOTAL TIME OF BOTH MOTION ESTIMATION STAGE AND VIDEO STABILIZATION STAGE

category	stabilized-encoder with EPZS				our method with EPZS			
	QP = 22	QP = 27	QP = 32	QP = 37	QP = 22	QP = 27	QP = 32	QP = 37
aerial	38.82%	35.11%	36.88%	34.97%	60.2%	57.79%	55.76%	58.26%
crowd	35.41%	34.62%	34.99%	32.74%	62.3%	61.86%	60.13%	61.73%
driving	37.64%	32.34%	33.59%	31.35%	52.08%	49.58%	51.73%	52.62%
helmet	38.15%	37.22%	39.08%	38.68%	62.43%	60.9%	60.24%	62.71%
parallax	32.71%	32.01%	34.39%	32.05%	52.05%	51.86%	51.57%	52.15%
pet	37.3%	35.86%	39.52%	34.17%	61.75%	60.74%	60.36%	60.99%
regular	27.2%	33.98%	39.8%	37.02%	55.91%	59.58%	59.42%	63.89%
rotation	21.76%	23.18%	27.98%	33.05%	45.03%	45.61%	53.9%	56.18%
running	22.21%	13.01%	30.83%	35.4%	52.7%	46.92%	60.27%	62.44%
zooming	22.41%	2.58%	34.37%	33.18%	50.01%	37.72%	57.99%	61.07%
Avg.	31.36%	27.99%	35.14%	34.26%	55.45%	53.25%	57.14%	59.2%
	32.19%				56.26%			

category	stabilized-encoder with UMHS				our method with UMHS			
	QP = 22	QP = 27	QP = 32	QP = 37	QP = 22	QP = 27	QP = 32	QP = 37
aerial	4.78%	7.79%	12.07%	14.17%	33.65%	33.02%	43.42%	38.45%
crowd	8.71%	11.3%	18.14%	19.86%	25.95%	24.13%	42.52%	32.24%
driving	13.37%	10.25%	17.05%	19.5%	35.53%	33.07%	47.5%	37.35%
helmet	20%	13.38%	27.04%	28.83%	27.58%	27.96%	30.24%	35.6%
parallax	-0.3%	-2.36%	10.14%	15.17%	17.13%	19.49%	31.17%	29.69%
pet	10.49%	7.02%	14.18%	19.76%	27.7%	24.53%	33.65%	36.87%
regular	18.49%	14.34%	22.97%	26.24%	26.97%	26.41%	33.24%	38.09%
rotation	9.38%	0.85%	15.42%	18.12%	20.03%	24.66%	34.13%	37.08%
running	18.94%	9.87%	20.56%	25.21%	19.63%	20.92%	33.1%	35.29%
zooming	12.48%	1.39%	15.65%	18.53%	24.61%	26.73%	42.87%	37.76%
Avg.	11.64%	7.38%	17.32%	20.54%	25.88%	26.09%	37.18%	35.84%
	14.22%				31.24%			

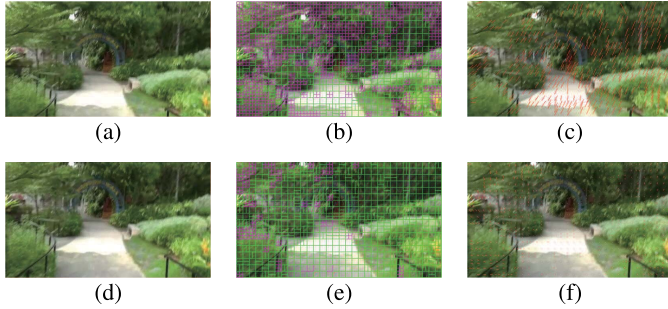


Fig. 12. Comparison between the original frame and the stabilized frame sampled from the category of *running*. (a) is the original shaky frame, with (b) and (c) shown in the block-mode and motion vectors. (d) is the stabilized frame, with (e) and (f) shown in the block-mode and motion vectors. The I-mode macroblocks are colored in purple in (b) and (e), and the motion vectors are colored in orange in (c) and (f).

coding effectiveness (PSNR, BD-PSNR, and BD-rate) with respect to stabilized-encoder, but we will see the computational cost having a significant reduction as below.

2) *Computational Cost*: The time saving (TS) is adopted to represent the computational cost decrement of our method and stabilized-encoder compared to shaky-encoder. The detailed TS for each category is shown in Table II. Here, we evaluate the cost of motion estimation by averaging the elapsed time of both motion estimation stage and video stabilization stage, which was done on a desktop computer with a 3.3 GHz Intel Quad Core CPU and 8GB RAM. It can be seen that our method reduces the computational cost about 56.3% with EPZS and 31.24% with UMHS compared to shaky-encoder,

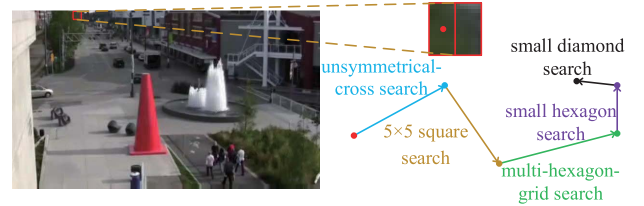


Fig. 13. A sub-macroblock with the size of 16×8 extracted from a sampled frame of a video in the category of *regular*, and the procedure of motion search in the ordinary encoder.

which is mainly because of the removal of shakiness and motion vector optimization proposed in Section V. The severe shakiness causes a large distance between the current block and the best-matched block, and the times of motion search also increase (in the case of early-termination applied motion search algorithm). Therefore, it can save the computational effort by removing those shakiness.

Meanwhile, our method also has much less computational cost than stabilized-encoder (24.1% with EPZS and 17% with UMHS). This is because the statistics based motion vector optimization also contributes to the reduction a lot (see the comparison between stabilized-encoder and our method in Table II). To prove this point, we have investigated the process of motion search of a 16×8 sub-macroblock of a video in the category of *regular*. We take the sub-macroblock located in the 8-th macroblock of the 5-th frame as an example (see Fig. 13). In the ordinary encoder, the initial point of this sub-macroblock is assumed to be (68, 36) (in the unit of

TABLE III
THE COMPUTATIONAL COST OF ITERATION TIMES FOR MOTION
SEARCH IN THE ORDINARY ENCODER

	Unsymmetrical-cross	5×5 Square	SDS
times	48	24	8
	Multi-hexagon-grid	Small hexagon	Overall
times	144	6	230

quarter-pixel) after the small diamond search. Then the motion vector is computed by using unsymmetrical-cross search, and the final motion vector is generated after 5×5 square search, multi-hexagon-grid search, small hexagon search and small diamond search sequentially. The cost of iteration times for each search procedure is shown in Table III, where the overall cost is up to 230 iterations. On the contrary, our method derives a better motion vector by just using SMV_s and DMV , where only one time of calculation is needed towards the desirable motion vector. Therefore, our method stands out for its efficiency in encoding the shaky videos, which has lower computational cost yet comparable compression quality to stabilized-coder as indicated by the R-D performance.

D. Experiments With Synthetic Data

To better demonstrate the effectiveness of our method, we also conduct the experiments with synthetic videos that have ground truth stable videos as the reference. This enables the evaluation of PSNR directly based on the ground truth data.

We can employ computer graphics rendering technique to produce the virtual scenes for the ground truth stable videos, as well as the shaky videos by adding noise on the camera path. But the rendered scenes are not such natural as the real world. So we follow the methodology of video quality assessment (VQA) that synthesizes the shaky videos from stable videos. Concretely, we collect 15 clips as the ground truth data, which are from movies, TV shows, etc. Such videos are typically captured by cameras with hardware stabilizers like tripod and dolly. Then, shaky videos are generated by adding synthetic jitter of different degrees on the ground truth videos as in [34]. We synthesize 4 kinds of jitter (J1, J2, J3, J4) with the severity of shakiness from slight to severe (see Fig. 14(a) and (b)). Thus, there are 4 shaky videos for each ground truth video, and totally 60 shaky videos in our synthetic dataset. These synthetic videos can be downloaded from our webpage.² Then, we encode these videos by our method with EPZS (QP = 27), shaky-encoder and stabilized-encoder as mentioned in Section VI-C.

Fig. 14(c) and (d) show the values of PSNR and bit rate by shaky-encoder, stabilized-encoder, and our method. Here, the PSNR values are computed with the ground truth data as the reference. It can be seen that our method can obtain almost the same performance with the stabilized-encoder, and both of them are superior to shaky-encoder. These results coincide with the ones obtained based on the real data in Section VI-C, while the computational cost of our method is still much less than the other two methods with the synthetic data.

²<https://zhangleibit.github.io/download/stabcoding/syntheticdata.html>

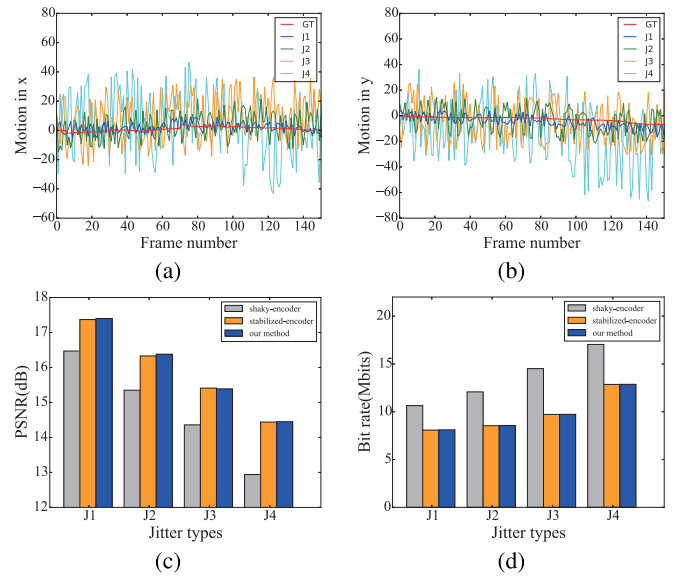


Fig. 14. The coding performance of shaky-encoder, stabilized-encoder and our method on the synthetic data with 4 kinds of jitter (J1, J2, J3, J4) added to the ground truth videos (GT). (a) The horizontal shakiness denoted by the x-coordinates of the shaky motion. (b) The vertical shakiness denoted by the y-coordinates of the shaky motion. (c) and (d) show the values of PSNR and bit rate of the three methods.

TABLE IV
COMPARISONS ON PSNR, BIT RATE AND COMPUTATIONAL COST OF
DIFFERENT METHODS ('+' MEANS RAISE AND '-' MEANS
REDUCTION WITH RESPECT TO THE ORDINARY ENCODER)

Methods	PSNR(dB)	Bit rate(%)	Computational cost(%)
method of [2]	-0.10 ~ -0.80	slightly	+12 ~ +15
SaVE/SC2 [14]	+0.01 ~ +0.42	n/a	-24.78
our method	+0.02 ~ +2.10	-19.87	-26.73

E. Comparisons

We also compared our method with other integrative method, rather than encoding the stabilized videos separately, in the aspects of both video coding efficiency and video stabilization quality.

1) *Coding Efficiency*: We investigate the performance of the methods of [2] and [14] on the PSNR, bit rate and computational cost in video coding. Table IV shows the comparison results, where the statistics are collected directly from the report in the paper for a fair judgement. Actually, the method of [2] proposes three schemes for the integration of video coding and stabilization. Here we choose the second scheme that is most related to our method for comparison, i.e., based on the reuse of motion vectors from stabilization. It can be seen that the computational cost increases due to the block-based motion estimation in stabilization. Meanwhile, the inaccuracy of the interpolated motion vectors slightly reduces the PSNR of the encoded videos. The method of [14], namely SaVE/SC2, can realise the increase of PSNR and the reduction of computation. But it leverages sensors like accelerometer and compass, to produce accurate motion vectors, while it does not give any report on the performance of bit rate.

In contrast, our method achieves a good balance on the optimization of not only PSNR and computational cost, but

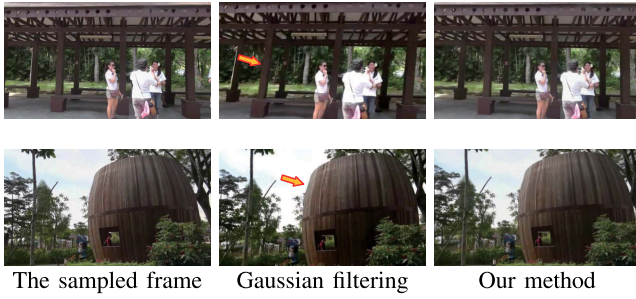


Fig. 15. Comparison of the stabilized frames produced by Gaussian filtering based stabilization and our method. The red arrows indicate distortions in the frames.

also bit rate after video coding. For PSNR and computational cost, our method enables better performance compared to the scheme in the method of [2]. Moreover, as a pure algorithm based system, our method has the comparable performance on the computational cost with sensor-assisted SaVE/SC2.

2) *Stabilization Quality*: The integration of geodesic video stabilization [12] in video coding not only boosts the coding efficiency, but also can improve the stabilization quality of the encoded videos. As previous integration methods prefer low-pass filtering in video stabilization [2], [3], we just make a comparison on the stabilization results obtained by low-pass filtering, e.g., Gaussian filtering as described in [2].

Generally, low-pass filtering cannot well stabilize shaky motion with large shakiness amplitude, especially for those videos in the categories of *helmet*, *pet*, *running*, etc. We refer the reader to the companion demo video for dynamic exhibition on the stabilization results. Besides, this sort of stabilization might incur geometric distortion in the stabilized frames. Fig.15 shows two sampled frames and their stabilized results. On the contrary, our method enables better results based on geodesic stabilization, which achieves both high-quality stability and less distortion in the stabilized videos.

F. Low-Latency Encoding

With the geodesic video stabilization, full HD (1920×1080) video can be processed at the speed up to 23.69 milliseconds (ms) per frame [12]. So it is possible that our integration can work at low-latency if we adopt a fast encoder in practice. Fortunately, an open-source-software named x264 can fulfill our expectation. So we also build a system based on x264 besides of the aforementioned implementation with the JM encoder.

We still adopt the system depicted in Fig.1 and just replace the JM encoder with x264 encoder. The x264 encoder follows the common video coding procedure, so the replacement is straightforward. And the reason for the use of x264 version rather than JM version is that x264 encoder can provide acceptable delay to fulfil the low-latency encoding in this experiment. Specially, we adopt two preset-modes by the encoder with a set of prescribed parameters in advance as described in Table V.

Based on the two modes, we have also implemented our system on a mobile device with a 2.4GHz ARM Cortex processor³ and 3G RAM, and tested it with the videos in our

³<https://www.96boards.org/product/hikey960/>

TABLE V

TWO PRESET-MODES IN X264 ENCODER WITH SOME KEY PARAMETERS SETTING FOR LOW-LATENCY ENCODING, AND THE RUNNING TIME (IN FPS) FOR HD AND FULL HD VIDEOS ON BOTH A DESKTOP COMPUTER (FPS1) AND AN ARM-BASED MOBILE DEVICE (FPS2)

Mode	Parameter setting		Running time		
	parameter	value	size	FPS1	FPS2
veryfast	reference frame number	1	HD	55.9	44.3
	mode decision	SATD	Full HD	27.5	25.3
ultrafast	reference frame number	1	HD	79.6	68.2
	adaptive spatial transform size	disabled	Full HD	34.9	32.4
	B-frame	no			
	motion search strategy	diamond			

dataset. Table V shows the timing of frames per second (FPS) for the integration system running in a multi-threaded environment, based on both desktop computer and mobile device. It can be seen that our integration can handle full HD video with the speed of more than 30 FPS with the *ultrafast* mode on the two platforms (see Table V), which can achieve low-latency encoding for the shaky videos.

G. Limitation and Discussion

The efficacy of our integrative method relies partly on the effectiveness of video stabilization. The coding procedure degenerates if the stabilizer generates inaccurate motion information. Besides, the *selective search strategy* that is used in Section VI-B may cause a slight difference in the bit rate and quality of the encoded videos.

Despite a few limitations, our integrative system achieves good performance in encoding shaky videos. Due to the use of geodesic video stabilization, sufficient motion vectors from feature points and transformations can be reused in the video coding. Though not always the ideally best motion vectors, they have a pretty good coincidence in statistics and enable very efficient motion estimation for video coding. Thus, our system can significantly reduce the computational cost of video coding.

VII. CONCLUSION

We have presented a novel method of integrating an efficient video stabilization method with video coding. For shaky videos, our integration produces more accurate motion vectors with lower computational cost than the ordinary encoder. Meanwhile, bit rate and PSNR also achieve comparable performance with directly encoding the stabilized videos. After the processing of our integrative system, stabilized videos with low bit rate can be efficiently encoded in a one-shot manner.

As the future work, we plan to extend our method to multiple reference frames, which are encoded by reusing the stabilization information of macroblocks at the same position to further improve the efficiency. Besides, it is also promising to implement the integration of video stabilization and coding based on GPU, which could further improve the efficiency of the integrative system.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their valuable comments.

REFERENCES

- [1] B. Lei, R. K. Gunnewiek, and P. H. N. de With, "Reuse of motion processing for camera stabilization and video coding," in *Proc. ICME*, Jul. 2006, pp. 597–600.
- [2] H. H. Chen, C. K. Liang, Y. C. Peng, and H. A. Chang, "Integration of digital stabilizer with video codec for digital video cameras," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 17, no. 7, pp. 801–813, Jul. 2007.
- [3] W.-H. Wang, Y.-S. Tsai, C.-L. Luo, and G.-R. Chen, "Fusion of video encoding and image stabilization," *Proc. SPIE*, vol. 5637, pp. 42–50, Feb. 2005.
- [4] M. Grundmann, V. Kwatra, and I. Essa, "Auto-directed video stabilization with robust L1 optimal camera paths," in *Proc. CVPR*, Jun. 2011, pp. 225–232.
- [5] F. Liu, M. Gleicher, J. Wang, H.-L. Jin, and A. Agarwala, "Subspace video stabilization," *ACM Trans. Graph.*, vol. 30, no. 1, pp. 4:1–4:10, Feb. 2011.
- [6] F. Liu, M. Gleicher, H.-L. Jin, and A. Agarwala, "Content-preserving warps for 3D video stabilization," *ACM Trans. Graph.*, vol. 28, no. 3, pp. 44:1–44:9, Aug. 2009.
- [7] Z. Zhou, H. Jin, and Y. Ma, "Plane-based content preserving warps for video stabilization," in *Proc. CVPR*, Jun. 2013, pp. 2299–2306.
- [8] S.-C. Liu, L. Yuan, P. Tan, and J. Sun, "Bundled camera paths for video stabilization," *ACM Trans. Graph.*, vol. 32, no. 4, pp. 78:1–78:10, Jul. 2013.
- [9] S. Liu, M. Li, S. Zhu, and B. Zeng, "Codingflow: Enable video coding for video stabilization," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3291–3302, Jul. 2017.
- [10] S.-C. Liu, P. Tan, L. Yuan, J. Sun, and B. Zeng, "Meshflow: Minimum latency online video stabilization," in *Proc. ECCV*, 2016, pp. 800–815.
- [11] J. Kopf, "360° video stabilization," *ACM Trans. Graph.*, vol. 35, no. 6, pp. 195:1–195:9, Nov. 2016.
- [12] L. Zhang, X.-Q. Chen, X.-Y. Kong, and H. Huang, "Geodesic video stabilization in transformation space," *IEEE Trans. Image Process.*, vol. 26, no. 5, pp. 2219–2229, May 2017.
- [13] J. Jain and A. Jain, "Displacement measurement and its application in interframe image coding," *IEEE Trans. Commun.*, vol. COM-29, no. 12, pp. 1799–1808, Dec. 1981.
- [14] X. Chen, Z. Zhao, A. Rahmati, Y. Wang, and L. Zhong, "Sensor-assisted video encoding for mobile devices in real-world environments," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 21, no. 3, pp. 335–349, Mar. 2011.
- [15] L. Ding, Y. Tian, H. Fan, Y. Wang, and T. Huang, "High-efficiency coding for shaking surveillance videos based on global motion compensation," in *Proc. BigMM*, Apr. 2016, pp. 259–265.
- [16] K.-Y. Lee, Y.-Y. Chuang, B.-Y. Chen, and M. Ouhyoung, "Video stabilization using robust feature trajectories," in *Proc. ICCV*, Sep. 2009, pp. 1397–1404.
- [17] S. Battiato, G. Gallo, G. Puglisi, and S. Scellato, "Sift features tracking for video stabilization," in *Proc. ICIAP*, Sep. 2007, pp. 825–830.
- [18] Y. Matsushita, E. Ofek, X. Tang, and H.-Y. Shum, "Full-frame video stabilization," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, Jun. 2005, pp. 50–57.
- [19] J. Zhou, H. Hu, and D. Wan, "Video stabilization and completion using two cameras," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 21, no. 12, pp. 1879–1889, Dec. 2011.
- [20] G. Puglisi and S. Battiato, "A robust image alignment algorithm for video stabilization purposes," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 21, no. 10, pp. 1390–1400, Oct. 2011.
- [21] F.-L. Zhang, J. Wang, H. Zhao, R. Martin, and S.-M. Hu, "Simultaneous camera path optimization and distraction removal for improving amateur video," *IEEE Trans. Image Process.*, vol. 24, no. 12, pp. 5982–5994, Dec. 2015.
- [22] S. Liu, Y. Wang, L. Yuan, J. Bu, P. Tan, and J. Sun, "Video stabilization with a depth camera," in *Proc. CVPR*, Jun. 2012, pp. 89–95.
- [23] C. Jia and B. L. Evans, "Constrained 3D rotation smoothing via global manifold regression for video stabilization," *IEEE Trans. Signal Process.*, vol. 62, no. 13, pp. 3293–3304, Jul. 2014.
- [24] Y.-S. Wang, F. Liu, P.-S. Hsu, and T.-Y. Lee, "Spatially and temporally optimized video stabilization," *IEEE Trans. Vis. Comput. Graphics*, vol. 19, no. 8, pp. 1354–1361, Aug. 2013.
- [25] L. Zhang, Q.-K. Xu, and H. Huang, "A global approach to fast video stabilization," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 27, no. 2, pp. 225–235, Feb. 2017.
- [26] B. Hall, *Lie Groups, Lie Algebras, and Representations: An Elementary Introduction*. New York, NY, USA: Springer, 2004.
- [27] E. Zacur, M. Bossa, and S. Olmos, "Left-invariant riemannian geodesics on spatial transformation groups," *SIAM J. Imag. Sci.*, vol. 7, no. 3, pp. 1503–1557, Jul. 2014.
- [28] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H. 264/AVC video coding standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.
- [29] G. J. Sullivan and T. Wiegand, "Rate-distortion optimization for video compression," *IEEE Signal Process. Mag.*, vol. 15, no. 6, pp. 74–90, Nov. 1998.
- [30] S. Zhu and K.-K. Ma, "A new diamond search algorithm for fast block-matching motion estimation," *IEEE Trans. Image Process.*, vol. 9, no. 2, pp. 287–290, Feb. 2000.
- [31] A. M. Tourapis, A. Leontaris, K. Suhring, and G. Sullivan, *H.264/14496-10 AVC Reference Software Manual*, document JVT-AE010, 2009.
- [32] Z. Chen, J. Xu, Y. He, and J. Zheng, "Fast integer-pel and fractional-pel motion estimation for H.264/AVC," *J. Vis. Commun. Image Represent.*, vol. 17, no. 2, pp. 264–290, Apr. 2006.
- [33] G. Bjontegaard, *Calculation of Average PSNR Differences Between RD-Curves*, document VCEG-M33, 2001.
- [34] H. Qu, L. Song, and G. Xue, "Shaking video synthesis for video stabilization performance assessment," in *Proc. Vis. Commun. Image Process. (VCIP)*, Nov. 2013, pp. 1–6.



Hua Huang (M'08) received the B.S. and Ph.D. degrees from Xi'an Jiaotong University, Xi'an, China, in 1996 and 2006, respectively. He is currently a Professor with the School of Computer Science, Beijing Institute of Technology, Beijing, China. His research interest is computer graphics and image and video processing.



Xiao-Xiang Wei received the B.S. degree in computer science from Beijing Institute of Technology, Beijing, China, in 2017, where he is currently pursuing the Ph.D. degree with the School of Computer Science. His research interest is image and video processing.



Lei Zhang (M'14) received the B.S. and Ph.D. degrees from Zhejiang University, Hangzhou, China, in 2004 and 2009, respectively. He is currently an Associate Professor with the School of Computer Science, Beijing Institute of Technology, Beijing, China. His research interests include computer graphics and image and video processing.